

The image is a screenshot of a web browser window displaying a chat application. The browser's address bar shows the URL 'claude.ai/chat/3727bab9-dc18-4e1b-95af-a090ed3991d9'. The chat interface has a dark theme. On the left, there is a sidebar with a vertical list of icons: a folder, a plus sign, a magnifying glass, a document, a link, a calendar, and a person. The main chat area shows a conversation. The user's message is: 'You are a Principal Software Engineer at OpenAI, Anthropic and Google. I have a Next.js 16 + React 19 + TypeScript AI Investment Research Platform called ReasonVest AI. I want you to refactor the ENTIRE backend architecture into production-quality code. DO NOT just fix bugs. Redesign it like a Senior Staff Engineer would. Goals: - Clean Architecture - SOLID Principles - DRY - Reusable Services - Future-proof - Enterprise-grade - Portfolio-worthy - Easy to extend Current APIs - Gemini - OpenRouter - Finnhub - Alpha Vantage - NewsAPI - MongoDB Current Features - AI Research - AI Chat Assistant - Company Analysis'. The AI's response is: 'Claude is AI and can make mistakes. Please double-check responses.' The bottom of the chat area has a text input field with the placeholder 'Write a message...' and a 'Send' button. The browser's bottom bar shows the system tray with the date '14-07-2024' and time '11:33'.

Voice-enabled AI chat application with Gemini-like Interface

AIError

ProviderError

ValidationError

8. Folder Structure

Redesign services folder.

9. Performance

Parallel requests

Caching where appropriate

Promise.all

Avoid duplicate API calls

10. Security

Validate every input

Never expose secrets

Sanitize prompts

11. Code Quality

Use latest TypeScript

Strict types

No any

No duplicated code

12. Documentation

Every function needs JSDoc

Explain WHY

not WHAT.

13. Generate complete files.

Never generate partial snippets.

Always output

FULL FILE

with filename

Write a message...

+

Sonnet 5 Extra





Claude is AI and can make mistakes. Please double-check responses.

33°C Sunny

  Search        

ENG IN

  

11:33 14-07-2026

Voice-enabled AI chat application with Gemini-like Interface

Share

with filename.
14. If another file must change because of architecture changes, tell me before generating.
I want production-ready code.
No shortcuts.
No toy examples.
Think like you're building software for millions of users.
Show less

10 Jul

Validated API response schema and commenced implementation planning

This is a substantial rewrite that touches almost every server-side file. Before I generate anything, here's the plan and exactly what changes:

Architecture plan

config/
env.ts
errors

- Zod-validated env, fails fast with readable errors

lib/
errors.ts
logger.ts
http.ts
into a safe JSON response
retry.ts

- AppError, ValidationError, ProviderError, AIError
- leveled logger
- toErrorResponse(): one place that turns any error into a safe JSON response
- retry(), fetchWithTimeout(), isRetryableStatus()

Write a message...

Sonnet 5 Extra

Claude is AI and can make mistakes. Please double-check responses.

33°C Sunny

Search

ENG IN

11:33 14-07-2026

Voice-enabled AI chat application

Invexa — Investment Decisions

Authjs | Errors

ChatGPT

Ask Gemini

claude.ai/chat/3727bab9-dc18-4e1b-95af-a090ed3991d9

Voice-enabled AI chat application with Gemini-like Interface

requestSchemas.ts — Zod schemas + parseRequest() for every route body

cache/memoryCache.ts — tiny in-process TTL cache

cache/reportCache.ts — two-tier (memory + optional Mongo) cache for full

research/reports

db/mongoClient.ts — lazy, connection-reusing Mongo client (no-op if MONGODB_URI unset)

services/ai/ — exactly the structure you specified

types.ts, prompts.ts, retry.ts, gemini.ts, openrouter.ts, providers.ts, aiService.ts

services/data/ — the SAME provider-agnostic pattern, applied to

market data

finnhub/{finnhubClient,companyService,profileService,financeService,newsService}.ts

alphaVantage.ts, newsApi.ts

companyDataService.ts, profileDataService.ts, financeDataService.ts, newsDataService.ts

(each: Finnhub primary → Alpha Vantage/NewsAPI automatic fallback)

services/research/

researchPipeline.ts — moved, now cached + typed + uses services/ai +

services/data

compareService.ts — extracted out of the compare route

One deliberate deviation from your file list, I added per your rule #14:
retry() / fetchWithTimeout() / isRetrivableStatus() are generic HTTP-resilience code, not

Write a message...

+ Sonnet 5 Extra

Claude is AI and can make mistakes. Please double-check responses.

33°C Sunny

Search

ENG IN

11:33 14-07-2026

Voice-enabled AI chat application with Gemini-like Interface

retry() / fetchWithTimeout() / isRetryableStatus() are generic HTTP-resilience code, not AI-specific — the data layer needs the exact same retry logic for Finnhub/Alpha Vantage/NewsAPI. So the real implementation lives in lib/retry.ts, and services/ai/retry.ts re-exports it — the file exists exactly where you asked, nothing that imports it changes, but the logic isn't duplicated. Same reasoning for lib/withFallback.ts (used by the data layer; the AI chain keeps its own loop since it needs per-provider retry-then-fallback plus provenance tracking, which is different enough to not force through the same helper).

Env vars: GEMINI_API_KEY and FINNHUB_API_KEY required (core features need them); OPENROUTER_API_KEY, ALPHAVANTAGE_API_KEY, NEWS_API, MONGODB_URI optional — each gates one fallback/feature and the app degrades gracefully without it, rather than refusing to boot.

Files deleted/moved (nothing else references them outside this list, confirmed by grep): services/aiService.ts, services/companyService.ts, services/profileService.ts, services/financeService.ts, services/newsService.ts, services/researchPipeline.ts (→ services/research/), lib/prompts.ts (→ services/ai/prompts.ts). All 4 API routes get rewritten. Zero frontend files change — I checked every consumer; the components only ever talked to the API routes over HTTP, never to these service files directly.

Building it now.

Edited a file, ran a command

Now the config layer:

Zod-validated environment configuration with read ↓ startup errors

Now the app boots

Write a message...

+ Sonnet 5 Extra

Claude is AI and can make mistakes. Please double-check responses.

33°C Sunny

Search

ENG IN

11:33 14-07-2026

Voice-enabled AI chat application with Gemini-like Interface

Application-wide error hierarchy - every error the app throws deliberately is one of these

Now the logger:

Leveled, emoji-prefixed logger for provider/fallback visibility in server logs

Now the generic retry/timeout utility:

Generic retry-with-backoff and timeout utilities, shared by every external API call in the app

resume the work

Verify current state before continuing

Good, state persisted. Continuing with `lib/http.ts`, `lib/withFallback.ts`, and `lib/requestSchemas.ts`:

Created 3 files

resume the work

Planted toward architectural sandstone layer foundations

Write a message...

+ Sonnet 5 Extra

Claude is AI and can make mistakes. Please double-check responses.

Voice-enabled AI chat application with Gemini-like Interface

+

🔍

📄

🔗

🔧

📱

🔊

📌

claude.ai/chat/3727bab9-dc18-4e1b-95af-a090ed3991d9

Ask Gemini

Share

Fully clean. Now the production build:

Validated API endpoints and error handling comprehensively

Now let's smoke-test the live routes end-to-end:

Smoke-test validation errors across all four routes

Now let's verify the full provider fallback chain behaves correctly with keys present but network calls failing (simulating a real outage):

Validated API fallback chains and error handling mechanisms thoroughly

Excellent — the full chain works exactly as designed: Gemini tried, got a real 403 (correctly classified non-retryable), immediately fell back to OpenRouter, which also failed, and the client got a safe, generic message while the server log has full diagnostic detail. Let's also verify the data-layer fallback and confirm Mongo caching no-ops gracefully:

Test data-layer fallback chain (Finnhub -> Alpha Vantage) and confirm no Mongo errors when unconfigur...

Good catch from that test — I need to fix a real bug: raw upstream status codes (like that 403 from a bad Finnhub key) were leaking through as *our* API's HTTP status, which is semantically wrong — a 403 on our endpoint implies the caller's request was forbidden, when actually it's our own server-to-server credential that failed. Let me fix this across all provider modules:

if im searching nvdia , applie , and microsoft it is working, if im searching

Write a message...

+

Sonnet 5 Extra

🔊

Claude is AI and can make mistakes. Please double-check responses.

33°C Sunny

Search

ENG IN

11:33 14-07-2026

That error is a perfect real-world illustration of exactly the two problems this refactor fixes:

1. **Raw upstream errors reaching the user** — that whole JSON dump is Google's raw error

Sonnet 5 Extra ▾  

Voice-enabled AI chat application with Gemini-like Interface

+

🔍

📁

🔗

📄

🔧

📱

Write a message...

+

Sonnet 5 Extra

🗣️

🔊

DashboardContent
file:///D:/reasonvest-ai-full%20(1)/.next/dev/static/chunks/_16t-uzh._js
(1571:231)
DashboardPage
file:///D:/reasonvest-ai-full%20(1)/.next/dev/static/chunks/_16t-uzh._js
(1422:221)
ClientPageRoot
file:///D:/reasonvest-ai-
full%20(1)/.next/dev/static/chunks/node_modules_next_dist_1e9_p6y._js
(4461:50) , how to get out of this , and tell also which companies can I search
and what can't , we also need to correct reply for the things if the company
does not having stock the AI needs to tell there is no stock with that and at
down it should mention there are some stocks with similar name and it needs
to give there information and also it should tell that if the company have sub
company investing those will get profit like that and the main was the AI
should give reply definitely and the GUI interface should be similar to gemini
AI interface like in the photo ,like same interface but ours needs to looks more
futuristic and mininmallistic

Show less

Check whether the project workspace persisted

Good, state persisted. Let's start with the crash — checking `FinancialCard.tsx` and every
place financial data gets rendered:

Identified null handling bugs in FinancialCard, auditing other components for similar issues

Claude is AI and can make mistakes. Please double-check responses.

33°C
Sunny

Search

🔍

📁

📅

📧

📺

📱

📄

🔧

📱

📄

🔧

📱

📄

🔧

ENG
IN

11:34
14-07-2026





